

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Bash Scripting and Automation

Unit 12 • Linux (10-151-105)



Learning Outcomes

This unit anchors two competencies: command-line administration (C2) and the scheduling side of processes, services, and logs (C5). By the end of the week you will be able to:

- Write Bash scripts with the sh-bang, comments, and sound naming conventions.
- Make scripts run — set execute permission and understand the role of \$PATH.
- Use variables, operators, conditionals, loops, and functions to add logic.
- Redirect streams and apply shell expansion to move data through a script.
- Schedule scripts to run automatically with cron and systemd timers.
- Apply secure-scripting habits so unattended automation can be trusted.

LESSON 1

Writing Bash Scripts



Why We Script

Administrators repeat the same tasks constantly — backups, updates, account creation, health checks. A script puts those commands in a file the system runs exactly as written, the same way every time, and can be scheduled to run on its own.

Advantage	What it buys you
Consistency	Runs the same way every time — no variability.
Speed	Executes commands far faster than you can type.
Accuracy	Once correct, it stays correct — no midnight typos.
Customization	Combine commands into tools tailored to your needs.
Scheduling	cron and systemd timers start it at any time you choose.

The Anatomy of a Script

Every Bash script opens with the interpreter directive — the “sh-bang” — telling the OS to run it with Bash. Comments (text after #) explain the code to humans and are ignored by Bash.

```
#!/bin/bash
# backup_home.sh - archive /home/student
echo "Starting backup..." # tell the operator
```

Naming conventions

- lowercase with _ or - (no spaces)
- avoid special characters
- lead with a verb: update_packages.sh
- .sh is optional but easy to spot

Linux identifies a file by its contents, not its extension — but .sh makes scripts easy to find with find or locate.

Making a Script Run

1. Set the execute permission.

Even your own script won't run until it carries the execute bit:

```
chmod +x backup_home.sh
```

2. Help Bash find it.

Bash only searches the directories in `$PATH` — and your home directory is not one of them. To run a script that lives there, prefix it with `./` (“look here”):

```
./backup_home.sh
```

Or move a finished script into a `$PATH` directory like `/usr/local/bin` so it runs by name. The same lesson returns under cron, where the minimal `$PATH` means you must use full paths.

LESSON 2

Script Building Blocks



Variables

A variable is a named container for a value. Assign with name=value (no spaces around =), and reference with a \$ in front of the name.

```
my_str="Hello, World!"  
echo "$my_str"      # prints: Hello, World!
```

Always quote values that contain spaces — on assignment and on use — to avoid word-splitting.

Shell vs. environment

A shell (local) variable lives only in the current shell. An environment variable is inherited by child processes — promote one with export.

Environment Variables & export

Use export to make a value visible to programs your shell launches:

```
export EDITOR="vim"    # inherited by child processes
```

Variable	Holds
\$HOME	Your home directory
\$USER	Your username
\$SHELL	Your login shell
\$PATH	Where Bash searches for commands

Persisting an export

- ~/.bash_profile — for your account
- a file in /etc/profile.d/ — system-wide

Operators

Operators let a script evaluate and compare values. Arithmetic goes in `$(( ))`; numeric comparisons inside `[]` use letter-based operators, not symbols.

Category	Operators
Arithmetic	+ - * / % (remainder)
Comparison (numeric)	-eq -ne -gt -ge -lt -le
Logical (Boolean)	&& (AND) (OR) ! (NOT)
Pattern	=~ match against a regular expression

Watch the trap: inside `[]`, write `[ $a -ge $b ]`, not `[ $a >= $b ]`. A single `=` assigns; comparisons use the operators above.

LESSON 3

Logic and Flow



Conditionals: if / elif / else

A conditional runs code only when a test is true. The if statement is closed by fi; add else for the false case and elif for further tests.

```
if [ $var -gt 10 ]
then
    echo "big"
elif [ $var -gt 1 ]
then
    echo "medium"
else
    echo "small"
fi
```

The test command ([])

checks conditions and returns true/false, including file tests:

-f file exists	-d directory exists
-r readable	-x executable

The case Statement

When you compare one value against many possibilities, case is cleaner than a stack of elif branches. It matches patterns, runs the matching block, and ends with esac. A final *) catches anything unmatched.

```
case $color in
  red)    echo "stop"   ;;
  green)  echo "go"     ;;
  *)      echo "unknown" ;;
esac
```

Double semicolons (;;) end each block. case stops at the first match — unlike a C-style switch, there is no fall-through.

Loops

A loop runs a block repeatedly. All three wrap their body between do and done; they differ in how they decide when to stop.

Loop	Runs the body...	Typical use
while	runs while a condition is true	Keep going until something changes
until	runs while a condition is false	Wait for a condition to become true
for	runs once per item in a list or range	Process every file, user, or value

```
for name in Kai Ali Joseph
do
    echo "$name is on the team."
done
```

A while loop needs a counter (or changing condition) inside it — otherwise it never ends.

Functions

A function is a named block of code you can call repeatedly — write the logic once instead of repeating it. Pass arguments as you would to a command and read them inside as \$1, \$2, and so on.

```
log_message() {  
    echo "[$(date)] $1"  
}  
  
log_message "Backup started"  
log_message "Backup finished"
```

Functions keep scripts readable and maintainable — which matters most when you revisit a script months later.

LESSON 4

Data and Streams



Redirection and Streams

Every command uses three streams: stdin (input), stdout (output), and stderr (errors). Redirection reroutes them — to files, or from one command into the next.

Operator	What it does
>	Send stdout to a file, overwriting it
>>	Send stdout to a file, appending
2>	Send stderr to a file (2>> appends)
&>	Send both stdout and stderr to one file
	Pipe — one command's output into another
<<	Here document — inline input until a marker

Scheduling pattern: `./backup.sh >> /var/log/backup.log 2>&1` captures output and errors alike.

Here Documents

A here document (<<) feeds several lines of inline text into a command until it reaches a chosen marker such as EOF — handy for sending a block of commands to a remote host or writing a small file from inside a script.

```
ssh user@host << EOF
  tar -cf backup.tar /etc
  date > /tmp/when.txt
EOF
```

The marker word (EOF here) can be anything — it just has to match top and bottom.

Shell Expansion

Before running a command, Bash expands special tokens into real values. The ones you'll use most:

Expansion	What it does
Command substitution	<code>\$(command)</code> drops a command's output into the line
Brace expansion	<code>\${word}s</code> appends text to a variable's value
Globbing	<code>* ? [abc]</code> match filenames (path expansion)
Positional parameters	<code>./report.sh march sales</code> → <code>\$1=march, \$2=sales</code>

LESSON 5

Scheduling Automation



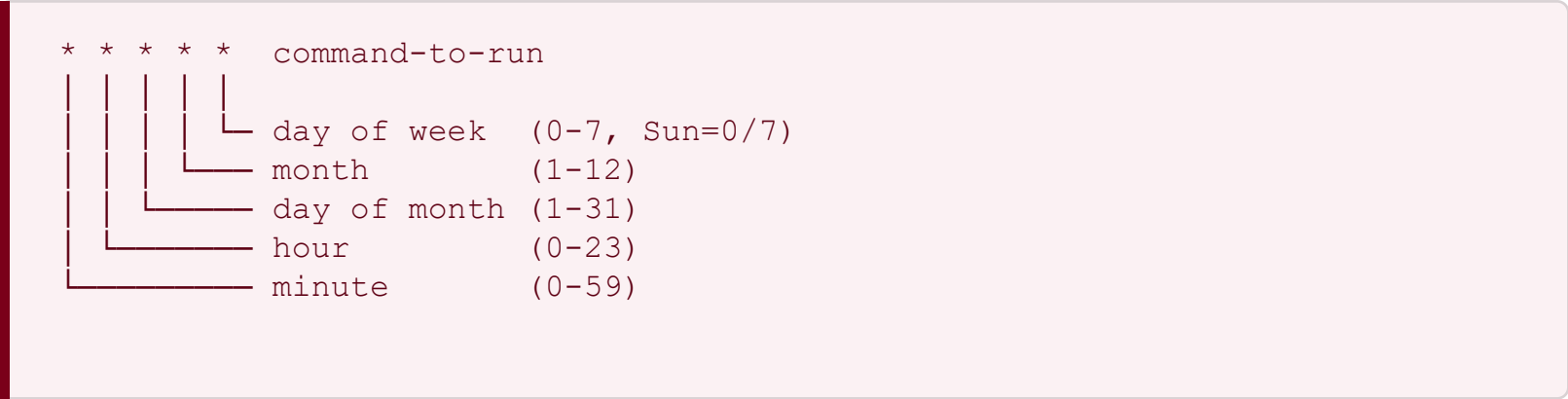
Why Schedule? Meet cron

A script you have to remember to run is only half-automated. cron is the classic Linux scheduler: a daemon that wakes every minute, checks a list of schedules, and runs whatever is due — with no one at the keyboard.

- A nightly backup at 2 a.m.
- An hourly health check
- A Monday-morning report

Each scheduled job is a single line in a crontab — five time fields plus the command to run.

The crontab: Five Fields



An asterisk means “every value.”
Shorthand strings also work:
@hourly @daily @weekly
@monthly @reboot

Schedule line	When it runs
0 2 * * *	2:00 a.m. every day
*/15 * * * *	every 15 minutes
30 6 * * 1	6:30 a.m. every Monday
0 0 1 * *	midnight on the 1st

Managing Crontabs

Command	What it does
crontab -e	Edit your crontab
crontab -l	List your jobs
crontab -r	Remove your crontab (no confirmation!)

Beyond your own crontab:

- /etc/crontab and /etc/cron.d/ — system schedules (add a user field)
- /etc/cron.{hourly,daily,weekly,monthly}/ — drop in an executable, no syntax needed

Jobs in your crontab run as you, with your permissions. Back up before editing: crontab -l > ~/mycron.bak

The cron Environment Trap

CAUTION

cron does not run with your interactive shell's environment

The `$PATH` is minimal and your home is the working directory. A script that runs perfectly when you type its name can silently fail under cron if it relies on a bare command name or a relative path.

The fix: use full, absolute paths to every command and file in a scheduled job — or set `PATH` at the top of the crontab. This single habit prevents the most common “but it works when I run it myself” cron failure.

systemd Timers

A second way to schedule on modern systems: a .service unit that does the work, paired with a .timer unit that says when.

```
# backup.timer
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
# enable the TIMER, not the service
systemctl enable --now backup.timer
systemctl list-timers
journalctl -u backup.service
```

Persistent=true runs a missed job at next boot. Timer behavior is identical on Rocky and Ubuntu — systemd is the unifier.

cron vs. systemd Timers

	cron	systemd timer
Logging	Mails output by default	Logs to the journal automatically
Missed runs	Skipped (anacron backstops)	Persistent= reruns at next boot
Best for	Quick, portable one-liners	Managed, observable, dependent jobs

Be fluent in both. cron is everywhere and portable; timers are cleaner to observe and manage at scale. A junior admin should read and use either when they inherit a system.

Also worth knowing: at runs a command once at a future time (atd service); anacron runs missed periodic jobs on machines that aren't always on.

Dual-Distro: the cron Service

Scripting and scheduling are among the most consistent areas of Linux — the crontab syntax, system locations, and systemd timers behave identically on both distributions. The one genuine difference is the service name, the same one from Unit 9:

Distribution	cron service	Package
Rocky Linux 10	crond.service	cronie
Ubuntu Server 24.04	cron.service	cron

Confirm it's running before expecting a job to fire: `systemctl status crond` / `systemctl status cron`

LESSON 6

Secure Scripting



Six Habits of Secure Scripting

A script runs commands automatically, often with privilege and unattended — so a careless one does damage fast. Build these habits in from the first line.

- 1. Quote your variables.** An unquoted `$var` is split on spaces and globbed. `rm $file` targets two files; `rm "$file"` does the right thing.
- 2. Validate input before you use it.** Untrusted until checked. The classic disaster: `rm -rf "$DIR/"` when `$DIR` is empty becomes `rm -rf /`.
- 3. Fail loudly and early.** Start serious scripts with `set -euo pipefail` so errors stop the script and unset variables are caught.

Six Habits (continued)

- 4. Never hard-code credentials.** A password or key in a script ends up in version control, backups, and process listings. Keep secrets in a protected file or a secrets store.
- 5. Use least privilege.** Don't run a whole script under sudo when one command needs root. Elevate narrowly, and use absolute paths for privileged commands.
- 6. Log what your scripts do.** An unattended script should leave an audit trail — a timestamped log file, or the journal via logger — so you can reconstruct what ran.

These connect forward to Ansible Vault (Unit 13) and back to the logging work of Unit 11.

Security Focus: cron as Persistence

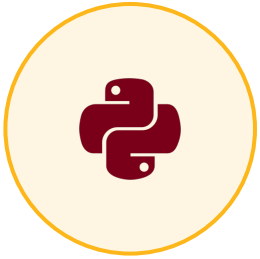


Why it matters

The same properties that make cron and timers useful — they run automatically, survive reboots, and execute as a chosen user — make them a favorite persistence mechanism for attackers. A single malicious crontab line or planted .timer can quietly re-establish access long after a break-in.

So auditing what is scheduled is itself a security skill: check every user's crontab (`crontab -l`), the system locations (`/etc/crontab`, `/etc/cron.d/`, the `cron.*` dirs), and `systemctl list-timers`. An entry you can't account for is exactly what a junior admin should flag.

A Note on Python



Bash is the native glue of Linux administration and the right tool for most day-to-day scripting. But it has a ceiling. When a task outgrows the shell — real data structures, parsing JSON or CSV, calling an API, sturdier error handling — the natural next step is Python.

They go hand in hand: many admins write Bash for quick automation and reach for Python when the logic gets complicated. `python3` ships by default on both Rocky 10 and Ubuntu 24.04.

AWARENESS

For this unit, Python is awareness-level only — recognize the kind of problem that warrants it, and know `python3` is available everywhere. Automation is a spectrum, from a one-line `crontab` entry to a maintained Python tool, and Bash scripting is where you start.

This Week's Lab

NETLAB+ Lab — Bash Scripting and Automation

- Write and run Bash scripts that take arguments, make decisions, and loop
- Schedule a script with both cron and a systemd timer
- Apply secure-scripting habits — quoting, validation, full paths, logging
- Perform every task on both Rocky Linux 10 and Ubuntu Server 24.04

Your work is graded on the observable end state — a script that runs, a job that's scheduled — not a description of it. Snapshot: WK12_SCRIPTING.

Key Takeaways

- A script is just commands in a file — but it brings consistency, speed, and the ability to schedule.
- Variables, conditionals, loops, and functions give a script logic; redirection and expansion move data through it.
- cron schedules with five time fields; systemd timers pair a .timer with a .service and log to the journal.
- Under cron, use absolute paths — the environment is minimal. The cron service is crond on Rocky, cron on Ubuntu.
- Secure scripting is a set of habits: quote, validate, fail early, no secrets, least privilege, log — and audit what's scheduled.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 12 · Bash Scripting and Automation

